

Allzweck-GPU- Programmierung mit CUDA

Vorlesung 1: Architektur,
Ausführungsmodell und der
erste Kernel

Torsten Bosse

6. Juli 2026

Lehrstuhl für Advanced Computing





Fahrplan dieser Vorlesung

Wir folgen dem Weg *vom Problem zum lauffähigen Kernel*:

- 1 **Warum GPUs?** – Datenparallelität, Latenz vs. Durchsatz.
- 2 **Hardware-Architektur** – Streaming-Multiprozessoren, Warps, SIMT, Speicherhierarchie.
- 3 **Programmiermodell** – Threads, Blöcke, Grid; Host und Device.
- 4 **Vom `for`-Loop zum Kernel** – die zentrale Denkfigur.
- 5 **Threads indizieren** – `threadIdx`, `blockIdx`, globaler Index, Grid-Stride.
- 6 **Speicher & Host-Ablauf.**
- 7 **Praxis** – Rechenumgebung der FSU, Übersetzen, Messen.

Anhang: konkrete GPUs und Architekturgenerationen.



- Erklären, *warum* sich GPUs für datenparallele Probleme eignen (Durchsatz statt Latenz, Latency Hiding);
- Hardware-Hierarchie *SM* → *Warp* → *Lane* und die Software-Hierarchie *Grid* → *Block* → *Thread* aufeinander abbilden;
- Datenparallele Schleife in einen CUDA-Kernel überführen und den globalen Thread-Index korrekt berechnen;
- Host-Ablauf (allokieren, kopieren, starten, zurückkopieren, freigeben) eigenständig schreiben;
- Kernel mit `nvcc` übersetzen, auf dem Cluster ausführen, die Laufzeit messen und Fehler prüfen.



Literatur & Quellen

- *CUDA C++ Programming Guide* –
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- *CUDA C++ Best Practices Guide* –
<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- Cheng, Grossman, McKercher, *Professional CUDA C Programming*, Wiley.
- Kirk & Hwu, *Programming Massively Parallel Processors*, Morgan Kaufmann.
- Williams, Waterman, Patterson, „Roofline: an insightful visual performance model“, *CACM* 52(4), 2009.
- NVIDIA-Architektur-Whitepapers (Pascal, Volta, Ampere, Hopper, Blackwell) –
<https://developer.nvidia.com>

Warum GPUs? – Die große Idee

GPGPU / CUDA – Vorlesung 1



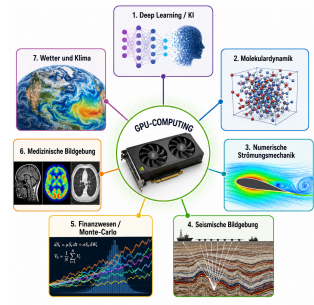
Motivation

Viele Anwendungen sind **datenparallel**:

- Bildverarbeitung (Pixel unabhängig)
- Simulationen (z. B. Partikelsysteme)
- Deep Learning (Matrixoperationen)

Idee:

- Zerlege Problem in (sehr) viele unabhängige Aufgaben
- Führe diese gleichzeitig aus



Beispiel: 1 Million Pixel bei Raytracer → 1 Million Threads

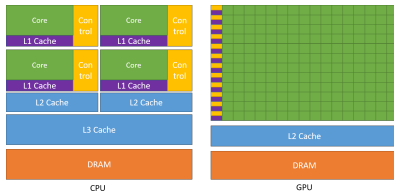


Die große Idee: Host + Device

Heterogenes Rechnen: CPU und GPU arbeiten zusammen.

- **Host (CPU)** – der *Dirigent*: serielle Logik, Steuerfluss, Ein-/Ausgabe, startet die Arbeit.
- **Device (GPU)** – das *Orchester*: tausende einfache Spieler führen denselben Part auf verschiedenen Daten aus.

Der Host *lagert* die rechenintensiven, datenparallelen Abschnitte als **Kernel** an das Device aus und holt die Ergebnisse zurück.



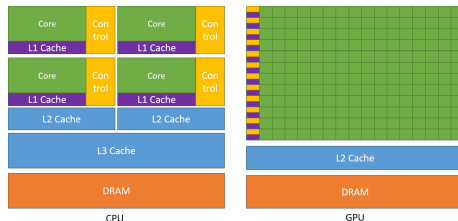


Hintergrund GPUs

- Seit zwei Jahrzehnten wächst die Single-Thread-Leistung nur langsam: Taktfrequenz und Instruktionsparallelität stagnieren.
- Die Transistorzahl steigt weiter $\Rightarrow$ nutzbar nur durch *explizite Parallelität*.
- GPUs ursprünglich extra Hardware für Bildverarbeitung mit fest verdrahtete 3D-Pipelines.
- GPUs bestehen aus sehr vielen einfachen Recheneinheiten, welche sehr viele Threads gleichzeitig ausführen.
- GPUs tauschen Single-Thread-Latenz gegen *Durchsatz*.
- Unified Shader Architekturen (ca. 2006) und APIs wie CUDA (ab 2007) wurden GPUs allgemein programmierbar (GPU $\Rightarrow$ GPGPU).



Entwurfsphilosophie: CPU vs. GPU



CPU (Latenz-Maschine)

- optimiert für **geringe Latenz**
- wenige, komplexe Kerne
- starke Kontrolllogik (Branch Prediction, Caches)

GPU (Durchsatz-Maschine)

- optimiert für **hohen Durchsatz**
- viele einfache Recheneinheiten
- hohe Parallelität statt komplexer Einzelthread-Optimierung

Beide Architekturen nähern sich jedoch zunehmend an.



Massive Parallelität & Latency Hiding

GPUs erreichen hohe Leistung durch:

- sehr viele gleichzeitig aktive Threads
- einfache Recheneinheiten
- Verbergen von Speicherlatenz (**Latency Hiding**)

Prinzip:

- Wenn ein Thread auf Speicher wartet → ein anderer Thread läuft weiter

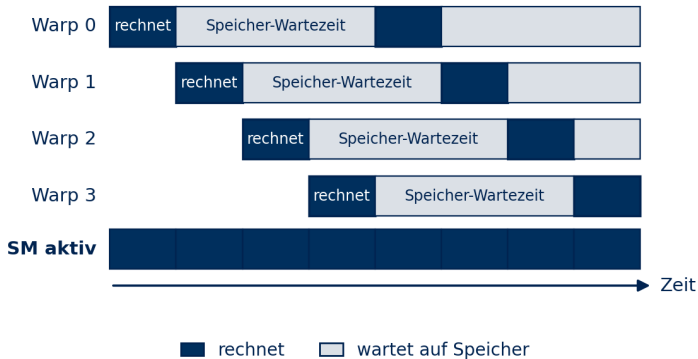
Konsequenz:

- hohe Auslastung der Hardware
- effiziente Nutzung der Speicherbandbreite



Latency Hiding veranschaulicht

Latency Hiding: wartet ein Warp auf Speicher, rechnet ein anderer



Die GPU hält viele Warps (=Gruppe von Threads) bereit: Wartet ein Warp auf den globalen Speicher, schaltet der Scheduler auf einen rechenbereiten Warp um. Device bleibt ausgelastet (Latenz wird durch Parallelität *verdeckt*, nicht beseitigt).



Wie viel bringt das? – Amdahlsches Gesetz

Nur der *parallelisierbare* Anteil $1 - f$ eines Programms profitiert von der GPU. Mit p parallelen Recheneinheiten ist der Speedup

$$S(p) = \frac{1}{f + \frac{1-f}{p}} \quad \xrightarrow{p \rightarrow \infty} \quad S_{\max} = \frac{1}{f}.$$

- Beispiel: $f = 0.05 \Rightarrow S_{\max} = 20$ – selbst mit beliebig vielen Kernen.
- Der serielle Rest f und die Datenübertragung Host $\leftrightarrow$ Device begrenzen den realen Gewinn.
- Konsequenz: rechenintensive, *stark* parallele Abschnitte auslagern; kleine, serielle Teile lohnen die Übertragung nicht.

GPU-Hardware-Architektur

GPGPU / CUDA – Vorlesung 1



Wesentliche Eigenschaften von GPGPUs

- Viele einfache Skalarprozessoren (SPs); keine komplexe Sprungvorhersage wie CPUs.
- SPs gruppiert in *Streaming-Multiprozessoren* (SMs), je eine breite SIMD/SIMT-Einheit.
- Sehr große Registersätze; Latenz durch Überzeichnung mit Threads verdeckt.
- Großer, aber langsamer globaler Speicher, hohe Latenz (hundert bis mehrere hundert Taktzyklen).
- Kleiner, schneller On-Chip-*Shared-Memory* (software-verwalteter Scratchpad).
- Caches klein im Vergleich zu CPUs.
- Historisch einfache Genauigkeit (FP32) $\gg$ doppelte Genauigkeit (FP64); heute zusätzlich FP16/BF16/FP8 für KI.



Aufbau einer modernen GPGPU

Eine GPGPU besteht aus vielen:

- **Streaming Multiprocessors (SMs)**

Jeder SM enthält:

- viele einfache Recheneinheiten (Skalarprozessoren, SPs)
- Register
- Shared Memory
- Warp-Scheduler

Wichtig: Ein SM führt viele Threads gleichzeitig aus.



Ein SM im Detail (moderner SM)



Ein typischer SM enthält:

- 4 Subpartitionen mit je eigenem Warp-Scheduler.
- FP32- + INT32-Lanes, weniger FP64-Einheiten, SFUs, Load/Store-Einheiten.
- Tensor-Cores für Matrix-Multiply-Accumulate (KI/HPC).
- Großer Registersatz (z. B. 65 536 32-Bit-Register).
- Konfigurierbaren L1 / Shared Memory (zehner bis hunderter kB).

Die genauen Zahlen variieren je Architektur; die *Struktur* ist das Entscheidende.

<https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>



Flynnnsche Klassifikation

	Single Instruction	Multiple Instruction
Single Data	klassische skalare CPU	fehlertolerante Systeme
Multiple Data	SSE/AVX	Mehrkern-CPU

GPUs implementieren eine SIMD-ähnliche Hardware, stellen dem Programmierer jedoch ein SIMT-(Software-)Modell zur Verfügung.



SIMT vs. SIMD

- **SIMT** = Single Instruction, Multiple Threads

Idee:

- viele Threads führen denselben Code aus
- Hardware führt sie gemeinsam (als Warp) aus
- Threads im Warp sind "im Gleichschritt"

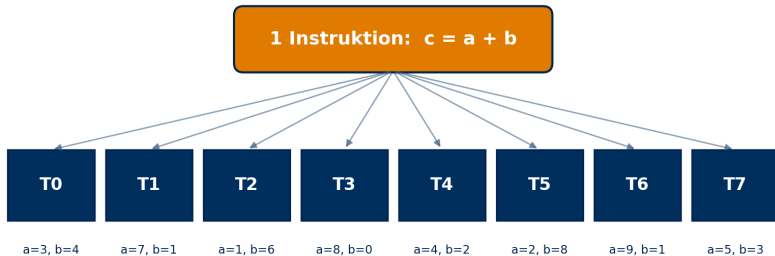
Abgrenzung:

- SIMD (CPU): ein Vektorregister, eine Lane-Gruppe
- SIMT (GPU): viele Threads mit eigener Identität (Index, Register)

Vgl. Flynn: GPUs sind *SIMD-Hardware* mit einem *SIMT-Programmiermodell* – die Threads dürfen logisch eigene Pfade nehmen.



SIMT: eine Instruktion, viele Lanes



alle Lanes führen dieselbe Instruktion aus — je eigene Register/Daten

Ein Warp = 32 Threads (hier 8 dargestellt)

Die *Lanes* eines Warps teilen sich ein Instruktionsfenster, besitzen aber je eigene Register. Datenparallelität „for free“, solange alle Lanes denselben Pfad nehmen.



Warps: die Ausführungseinheit der GPGPU

Threads werden in Gruppen zusammengefasst:

- **Warp = 32 Threads**

Eigenschaften:

- alle Threads eines Warps führen dieselbe Instruktion aus
- unterschiedliche Daten pro Thread

Problem: Divergenz

- unterschiedliche Kontrollpfade → Pfade werden serialisiert → langsamer

Beispiel: `if (threadIdx.x % 2) A(); else B();`
gerade und ungerade Lanes laufen *nacheinander* → halbe Auslastung.



Warp-Divergenz veranschaulicht



beide Pfade werden serialisiert → ~2× Laufzeit, halbe Auslastung

■ aktiv (rechnet) ▨ maskiert (untätig)

Nimmt ein Teil des Warps den `if`-Pfad und der Rest den `else`-Pfad, führt die Hardware beide *nacheinander* aus; die jeweils nicht beteiligten Lanes sind maskiert. Verzweigungen daher möglichst an Warp-Grenzen ausrichten.

Das CUDA-Programmiermodell

GPGPU / CUDA – Vorlesung 1



Von der Software zur Hardware

Wie wird Parallelität ausgedrückt? CUDA bietet eine Software-Hierarchie, die der Scheduler auf die Hardware abbildet:

Software (CUDA)	Hardware
Thread	Lane einer SM-Subpartition
Warp (32 Threads)	32 Lanes im Gleichschritt (SIMT)
Block	resident auf <i>einem</i> SM
Grid	verteilt über die SMs der GPU

- Es werden *sehr viele*, kurzlebige Threads gleichzeitig gestartet; der Scheduler bildet die Hierarchie *Thread* → *Warp* → *Block* → *Grid* auf *Lane* → *32-Lane-SIMT-Gruppe* → *SM* → *GPU* ab.
- Trennung macht denselben Code von kleinen bis großen GPU skalierbar.
- Programmierstandards: **CUDA** (NVIDIA, unser Fokus), OpenCL/SYCL (herstellernerneutral), OpenACC/OpenMP-Target.

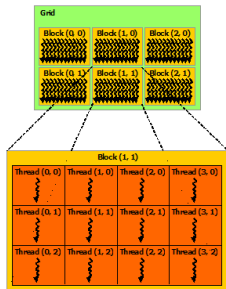


CUDA – Compute Unified Device Architecture

- Programmierung in C/C++ mit Erweiterungen; GPU-Funktionen sind *Kernels* (Function-Offloading).
- Der Compiler `nvcc` trennt den Quelltext in *Host-Code* (CPU, Standard-C++) und *Device-Code* (GPU).
- Funktions-Qualifizierer:
 - `__host__` – läuft auf der CPU, vom Host aufgerufen (Standard).
 - `__device__` – läuft auf der GPU, vom Device aufgerufen.
 - `__global__` – ein Kernel: läuft auf der GPU, vom Host gestartet.
- Quelldateien verwenden die Endung `.cu`.



Threads, Blöcke, Grid

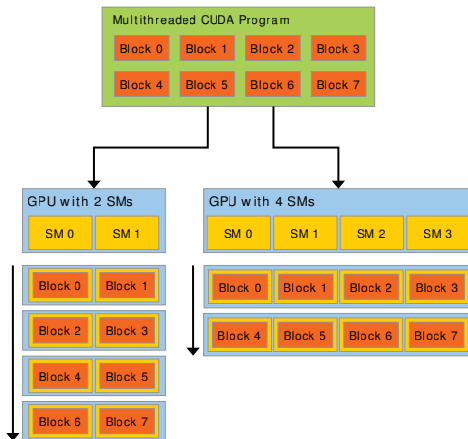


<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

- **Threads:** in 1-, 2- oder 3-D angeordnet.
- **Block:** bis zu 1024 Threads; synchronisierbar (`__syncthreads()`) und mit gemeinsamem *Shared Memory*; 32 Threads bilden einen Warp.
- **Grid:** die Menge aller Blöcke; bis zu $2^{31} - 1$ Blöcke pro Dimension.
- **Eingebaute Variablen:** `threadIdx`, `blockIdx`, `blockDim`, `gridDim`.
- **Globaler Index (1D):**
$$i = blockIdx.x * blockDim.x + threadIdx.x$$



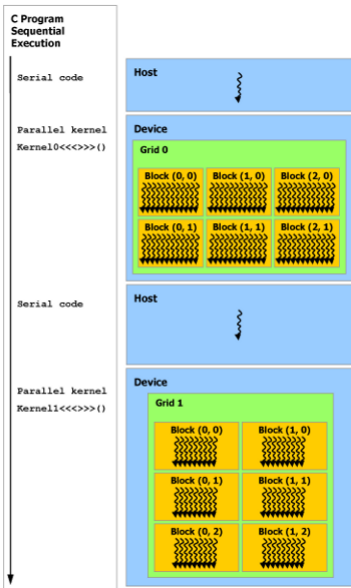
Abbildung der Blöcke auf SMs



Blöcke sind unabhängig und können in beliebiger Reihenfolge laufen
⇒ Automatische Skalierung von kleinen zu großen GPUs.



Heterogener Ausführungsablauf



Serielle Arbeit verbleibt auf der CPU;
Rechenintensive datenparallele
Arbeit wird als Kernel auf die GPU
ausgelagert.

Host und Device wechseln sich ab: serielle
Phase → Kernel-Start → parallele Phase →
Rückgabe der Kontrolle.

Vom `for`-Loop zum Kernel

GPGPU / CUDA – Vorlesung 1



Die serielle Ausgangsbasis (CPU)

Speichere jedes Element von a , um eins erhöht, in b :

```
int increment(int a) { return a + 1; }

void host(const int *a, int *b, int n) {
    for (int i = 0; i < n; ++i)
        b[i] = increment(a[i]);
}
```

Eine klassische datenparallele Schleife: jede Iteration ist unabhängig.



Seriell $\rightarrow$ Parallel

Eine parallele Schleife $\Rightarrow$ ein Thread pro Iteration

Seriell (CPU)

- Eine Schleife läuft `size`-mal *nacheinander*.
- Der Schleifenzähler `i` wählt das Element.

Parallel (GPU)

- Starte `size` Threads *gleichzeitig*.
- Jeder Thread *berechnet sein eigenes* `i` aus seiner Identität.
- Die Schleife verschwindet – ihr Rumpf wird zum Kernel.

Frage, die jeden Kernel beantwortet: „*Welches Element bearbeite ich?*“



Dasselbe als Kernel

```
// auf Host und Device nutzbar
__host__ __device__ int increment(int a) { return a + 1; }

__global__ void kernel(const int *a, int *b, int n) {
    int tid = threadIdx.x;           // Thread-Index im Block
    int bid = blockIdx.x;           // Block-Index im Grid
    int bdim = blockDim.x;          // Threads pro Block
    int i = tid + bid * bdim;        // globaler Index
    if (i < n)                       // Schutz: Grid kann zu gross sein
        b[i] = increment(a[i]);
}
```

Die Schleife entfällt: jeder Thread führt *eine* Iteration aus. Der Schutz `i < n` behandelt den letzten, nur teilweise gefüllten Block.



Kernel starten: die Ausführungskonfiguration

Die Tripelklammern `<<< . . . >>>` legen die *Geometrie* des Grids fest, d.h. *wie viele* Blöcke und *wie viele* Threads je Block:

```
int threads = 256; // Threads pro Block
int blocks  = (n + threads - 1) / threads; // Division mit Aufrundung
kernel<<<blocks, threads>>>(d_a, d_b, n);
```

- `blocks × threads ≥ n`; Aufrundungsdivision deckt auch nicht teilbares `n` ab.
- `threads` als Vielfaches der Warp-Größe 32 wählen (typisch 128/256/512) – sonst bleiben Lanes ungenutzt.
- Der überzählige letzte Block wird im Kernel mit `if (i < n)` abgefangen (siehe vorige Folie).
- Vollform `<<<blocks, threads, smBytes, stream>>>`: dynamischer Shared Memory und Streams.

Threads indizieren

GPGPU / CUDA – Vorlesung 1



Der globale Index – Schritt für Schritt

Beispiel: `blockDim.x = 256`. Jeder Block deckt 256 aufeinanderfolgende Elemente ab.

<code>blockIdx.x</code>	<code>threadIdx.x</code>	Rechnung	globaler Index <code>i</code>
0	0	$0 \cdot 256 + 0$	0
0	255	$0 \cdot 256 + 255$	255
1	0	$1 \cdot 256 + 0$	256
2	5	$2 \cdot 256 + 5$	517
<i>b</i>	<i>t</i>	$b \cdot 256 + t$	...

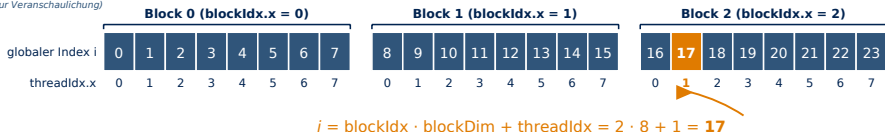
```
int i = blockIdx.x * blockDim.x + threadIdx.x;    // 1D-Standardformel
```

Jeder Thread erhält ein eindeutiges `i` – ohne Absprache, allein aus seiner Position im Grid.



Globaler Index – veranschaulicht

hier `blockDim = 8`
(zur Veranschaulichung)



Jeder Block deckt `blockDim` aufeinanderfolgende Indizes ab; die Formel `blockIdx*blockDim + threadIdx` weist jedem Thread genau ein Element zu (hervorgehoben: Block 2, Thread 1 $\rightarrow i = 17$).



2D- und 3D-Indexierung

Bilder und Matrizen sind natürlicherweise 2D, entsprechendes sollte für das Grid gelten:

```
dim3 threads(16, 16);           // 256 Threads pro Block
dim3 grid((W + 15) / 16, (H + 15) / 16); // genug Bloেকে fuer WxH
```

```
__global__ void invert(unsigned char *img, int W, int H) {
    int x = blockIdx.x * blockDim.x + threadIdx.x; // Spalte
    int y = blockIdx.y * blockDim.y + threadIdx.y; // Zeile
    if (x < W && y < H)
        img[y * W + x] = 255 - img[y * W + x]; // 2D -> 1D
        linearisiert
}
```

Das Muster bleibt identisch - nur je $.x$, $.y$ (und ggf. $.z$), wobei man auf *Schutz* in jeder Dimension achten muss.

Wenn die Daten größer sind als das Grid:



Grid-Stride-Loop

Häufig gibt es mehr Elemente als Threads. Robustes Idiom: jeder Thread bearbeitet *mehrere* Elemente im Abstand der Grid-Größe.

```
__global__ void scale(float *a, float s, int n) {  
    int i      = blockIdx.x * blockDim.x + threadIdx.x;  
    int stride = blockDim.x * gridDim.x;    // = Gesamtzahl Threads  
    for (; i < n; i += stride)  
        a[i] *= s;  
}
```

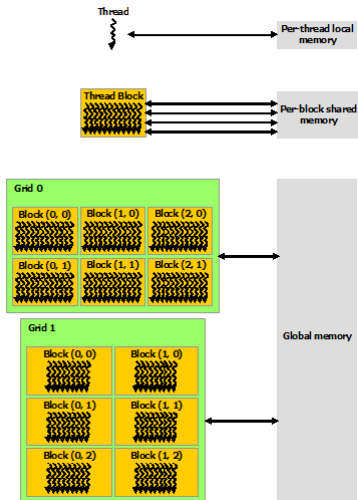
- Entkoppelt die Problemgröße n von der Startkonfiguration.
- Funktioniert für jede Grid-Größe – ein Thread, ein Block oder Tausende.
- Begünstigt *coalesced* Zugriffe.

Speichermodell & Host-Ablauf

GPGPU / CUDA – Vorlesung 1



Speicherhierarchie (das mentale Modell)



- **Register** – pro Thread, am schnellsten.
- **Shared Memory** – pro Block, On-Chip-Scratchpad.
- **Globaler Speicher** – Device-DRAM, groß, hohe Latenz.
- **Konstanten / Textur** – gecachte, nur-lesende Pfade.
- **Host-Speicher** – eigener Adressraum; Daten müssen übertragen werden (oder Unified Memory).

Die Kunst der CUDA-Performance besteht darin, Daten *oben* in dieser Hierarchie zu halten.



Host-seitige Schritte eines Kernel-Aufrufs

- 1 Device-Speicher allokalieren – `cudaMalloc()`.
- 2 Eingabe Host → Device kopieren – `cudaMemcpy(..., cudaMemcpyHostToDevice)`.
- 3 Kernel mit Ausführungskonfiguration starten:
`kernel<<<grid, threadsPerBlock, smBytes,
stream>>>(...)`
(nur `grid` und `threadsPerBlock` sind erforderlich).
- 4 Ergebnisse Device → Host kopieren – `cudaMemcpy(..., cudaMemcpyDeviceToHost)`.
- 5 Device-Speicher freigeben – `cudaFree()`.

Eselsbrücke: *allokalieren – kopieren – starten – zurückkopieren – freigeben.*



Vollständiges Host-Programm

```
int main() {
    int n = 1024;
    int *a_host, *b_host, *a_dev, *b_dev;
    a_host = (int*)malloc(size*sizeof(int));
    b_host = (int*)malloc(size*sizeof(int));
    fillA(a_host, n);

    cudaMalloc((void**)&a_dev, n*sizeof(int));
    cudaMalloc((void**)&b_dev, n*sizeof(int));
    cudaMemcpy(a_dev, a_host,n*sizeof(int), cudaMemcpyHostToDevice);

    dim3 threads(256);
    dim3 grid((size + threads.x - 1) / threads.x);    // Aufrundungsdivision
    kernel<<<grid, threads>>>(a_dev, b_dev, n);

    cudaMemcpy(b_host, b_dev,n*sizeof(int), cudaMemcpyDeviceToHost);
    checkResult(a_host, b_host,n);

    cudaFree(a_dev); cudaFree(b_dev);
    free(a_host); free(b_host);
    return 0;
}
```



Vereinfachung: Unified Memory

Statt getrennter Host-/Device-Puffer und expliziter Kopien genügt *ein* Zeiger; das System migriert die Daten bei Bedarf automatisch.

```
int *a;
cudaMallocManaged(&a, n*sizeof(int));    // von Host UND Device
      adressierbar
fillA(a, n);                             // Host-Zugriff
kernel<<<blocks, threads>>>(a, a, n);    // Device-Zugriff
cudaDeviceSynchronize();               // vor erneutem Host-Zugriff!
checkResult(a, n);
cudaFree(a);
```

- Weniger Code, weniger Fehlerquellen – ideal für Einstieg und Prototypen.
- Vorsicht: implizite Migration kann Performance kosten; für maximale Kontrolle bleibt explizites `cudaMemcpy` (oft mit Streams).

Rechenumgebung an der FSU

GPGPU / CUDA – Vorlesung 1



GPUs am Rechenzentrum der FSU

Ara-Cluster

- Nvidia Tesla P100
- Nvidia Tesla V100
- Nvidia Tesla A100

Draco-Cluster

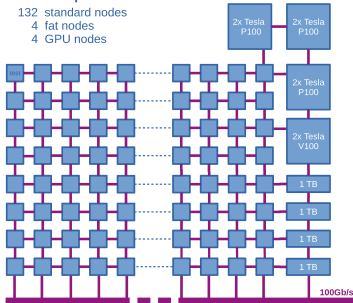
- Nvidia Tesla V100
- Nvidia Tesla A100



Aufbau ARA-Cluster

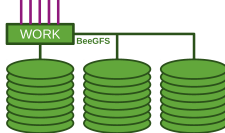
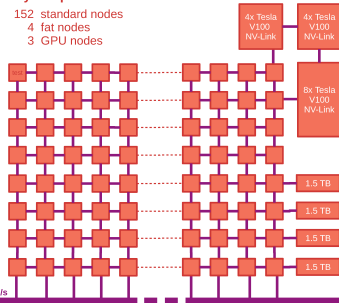
Broadwell partition

- 132 standard nodes
- 4 fat nodes
- 4 GPU nodes



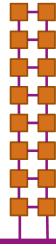
Skylake partition

- 152 standard nodes
- 4 fat nodes
- 3 GPU nodes



Hadoop

- 16 nodes



<https://wiki.uni-jena.de/spaces/URZ010SD/pages/22453005>

(c) A. Sternbeck, 2019



DRACO-Cluster



<https://confluence.uni-jena.de/spaces/URZ010SD/pages/109384499/HPC-Cluster+Draco>



Zugriff auf Ressourcen

■ Login (nach Freischaltung):

```
ssh <urz-kuerzel>@login1.ara.uni-jena.de  
sinfo  
sbatch script.sh
```

■ Information über die Grafikkarten abrufen (script.sh):

```
#!/bin/bash  
#SBATCH --partition=gpu_test  
#SBATCH --gres=gpu:1  
module load nvidia/cuda/12.4  
nvidia-smi
```

Übersetzen, Synchronisieren, Messen

GPGPU / CUDA – Vorlesung 1



Übersetzen und Ausführen

- Beispiel ist auf Moodle (Codes-CUDA-Beispiel)

- Übersetzen mit Make bzw. direkt via `nvcc`:

```
nvcc -o Beispiel Beispiel.cu -arch=sm_80
```

- `.cu`-Dateien dürfen Host- und Device-Code enthalten.
- `-arch=sm_80` erzeugt Code für Compute Capability 8.0 (Ampere, A100); P100 = `sm_60`, V100 = `sm_70`, A100 = `sm_80`.
- Register-/Shared-Memory-Verbrauch: `nvcc ... -Xptxas -v`.
- Ausführen mittels Slurm auf geeigneter Ressource.

<https://en.wikipedia.org/wiki/CUDA>



Synchronisation – wer wartet auf wen?

- Kernel-Starts sind **asynchron**: der Host läuft sofort weiter.
- `__syncthreads()` – Barriere für alle Threads *eines Blocks*; nötig vor dem Lesen gemeinsam beschriebener Daten.
- *Keine* eingebaute Barriere *über Blöcke hinweg* – globale Synchronisation entsteht erst durch das *Ende eines Kernels*.
- `cudaDeviceSynchronize()` – der Host wartet auf die Fertigstellung des Device.
- `cudaMemcpy` synchronisiert implizit (blockiert, bis fertig).

```
kernel<<<grid, threads>>>(a_dev, b_dev,n);  
cudaDeviceSynchronize();    // Host wartet, bis Kernel fertig ist
```



Zeitmessung auf der GPU mit Events

Für Zeitmessungen `cudaEvent_t` verwenden; Kernel-Aufrufe sind *asynchron*, daher ist eine WallClock-Messung auf dem Host irreführend.

```
float ms;
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start);
kernel<<<grid, threads>>>(a_dev, b_dev,n);
cudaEventRecord(stop);
cudaEventSynchronize(stop);           // auf GPU-Fertigstellung warten
cudaEventElapsedTime(&ms, start, stop); // verstrichene Zeit in ms
```



Fehlerbehandlung

- Jeder CUDA-Host-Aufruf liefert ein `cudaError_t` (`0 = cudaSuccess`).
- Kernel-Aufrufe sind asynchron – prüfen mit `cudaGetLastError()` (Aufruffehler) und einem synchronisierenden Aufruf (Ausführungsfehler).

```
kernel<<<grid, threads>>>(a_dev, b_dev,n);  
cudaError_t err = cudaGetLastError();  
if (err != cudaSuccess)  
    printf("CUDA-Fehler: %s\n", cudaGetErrorString(err));
```

In ein Makro kapseln, damit jeder Aufruf geprüft wird – stilles Versagen ist die häufigste Anfängerfalle.



Häufige Anfängerfehler

- **Host- und Device-Zeiger verwechseln** – ein Device-Zeiger darf auf dem Host nicht dereferenziert werden (und umgekehrt).
- **Fehlendes Error-Checking** – Kernel scheitern *still*; `cudaGetLastError()` nach jedem Start prüfen.
- **Vergessene Synchronisation** – der Host liest Ergebnisse, bevor der Kernel fertig ist (`cudaDeviceSynchronize()` bzw. implizit über `cudaMemcpy`).
- **Index außerhalb der Grenzen** – der `if (i < n)`-Schutz fehlt, der letzte Block läuft über.
- **Falsche Grid-Größe** – Ab- statt Aufrunden lässt die letzten Elemente unbearbeitet.
- **Annahme über die Block-Reihenfolge** – Blöcke laufen in *beliebiger* Reihenfolge; keine blockübergreifende Synchronisation.

Zusammenfassung

GPGPU / CUDA – Vorlesung 1



Kernaussagen

- GPUs sind Durchsatz-Maschinen: viele einfache Kerne, Latenz durch massives Multithreading verdeckt.
- CUDA legt eine Hierarchie offen: *Thread* → *Warp* → *Block* → *Grid*, abgebildet auf SMs.
- Ein Kernel ersetzt eine datenparallele Schleife; jeder Thread fragt „Welches Element bearbeite ich?“ und berechnet sein *i* selbst.
- Der Host orchestriert: allokieren, kopieren, starten, zurückkopieren, freigeben.
- Stets Fehler prüfen, mit Events messen, sauber synchronisieren.

Anhang: Konkrete GPUs & Architekturgenerationen

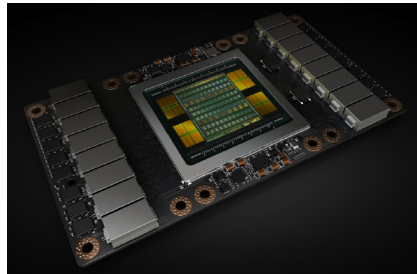
GPGPU / CUDA – Vorlesung 1



Typen von Grafikkarten



Geforce Kepler
(Endnutzer)



Tesla Volta
(Server)



Architekturgenerationen (zur Einordnung)

- 2012 Kepler – Dynamic Parallelism, großer Sprung bei Leistung/Watt.
- 2016 Pascal – HBM2, Unified Memory, NVLink, FP16.
- 2017 Volta – Tensor-Cores, unabhängiges Thread-Scheduling.
- 2018 Turing – RT-Cores, eigener Integer-Datenpfad neben FP.
- 2020 Ampere – Tensor-Cores der 3. Generation, asynchrones Kopieren.
- 2022 Hopper – Transformer Engine, Thread-Block-Cluster, FP8.
- 2024 Blackwell – FP4, Transformer Engine der 2. Generation.

Die *Compute Capability* (z. B. `sm_80`) kennzeichnet den Funktionsumfang einer GPU und das Ziel, für das `nvcc` übersetzen soll.

https://en.wikipedia.org/wiki/Nvidia_Tesla



Nvidia Kepler GK110 - Zahlen

- 7.1 Mrd. Transistoren (28nm)
- Daten für Maximalausbau in Tesla K40 (damals $\approx$ 6500 EUR)
 - 2880 Skalarprozessoren (SPs) (15 Multiprozessoren (SMs) x 192 SPs)
 - 12 GB GDDR5 RAM mit 384 Bit Speicherinterface (288 GB/s)
 - Performance bis zu 4290 GFlops (Single Precision)
 - Double Precision 1430 GFlops
- “Kleinster” GK110 in Geforce GTX 780 (damals $\approx$ 500 EUR)
 - 2304 SPs (12 Multiprozessoren)
 - 3 GB GDDR5 RAM (288 GB/s)
 - 3976 GFlops (SP), 165 GFlops (DP)



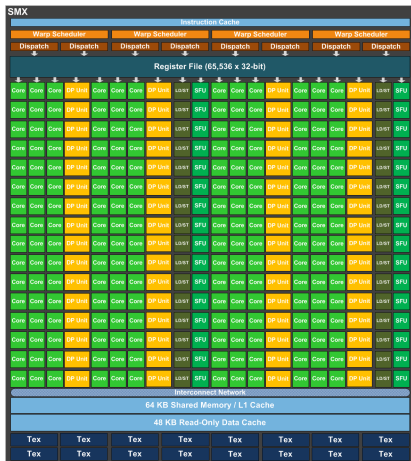
Nvidia Kepler GK110 - Struktur



NVIDIA Kepler GK110 Whitepaper (2012)



Nvidia Kepler GK110 - Struktur SMX



NVIDIA Kepler GK110 Whitepaper (2012)

- 192 SP Cores
- 64 DP Units
- 32 Load/Store Units (LD/ST)
- 32 Special Function Units (SFU)
- 4 Warp Scheduler (je 2 Dispatch)
- 65536 32-Bit Register
- 64 kB Shared Memory / L1 Cache
- 48 kB Read-Only Cache



Nvidia Blackwell B100 - Features



AI SUPERCHIP
208B Transistors



2nd GEN TRANSFORMER ENGINE
FP4/FP6 Tensor Core



5th GENERATION NVLINK
Scales to 576 GPUs



RAS ENGINE
100% In-System Self-Test



SECURE AI
Full Performance
Encryption & TEE



DECOMPRESSION ENGINE
800 GB/sec

[https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/
nvidia-rtx-blackwell-gpu-architecture.pdf](https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf)
<https://resources.nvidia.com/en-us-blackwell-architecture>